

Concurrent Programming

Outline

- Topics:
 - Concurrent programming with processes
 - Concurrent programming with threads
- Suggested reading:
 - 12.1~12.3

Concurrency

- Concurrency is a general phenomenon
 - Hardware exception handlers
 - Processes
 - Unix signal handlers

Concurrency

- Application-level concurrency is useful
 - Responding to asynchronous events
 - Computing in parallel on multiprocessor
 - Accessing slow I/O devices
 - Interacting with humans
 - Servicing multiple network clients
 - Agent exploration

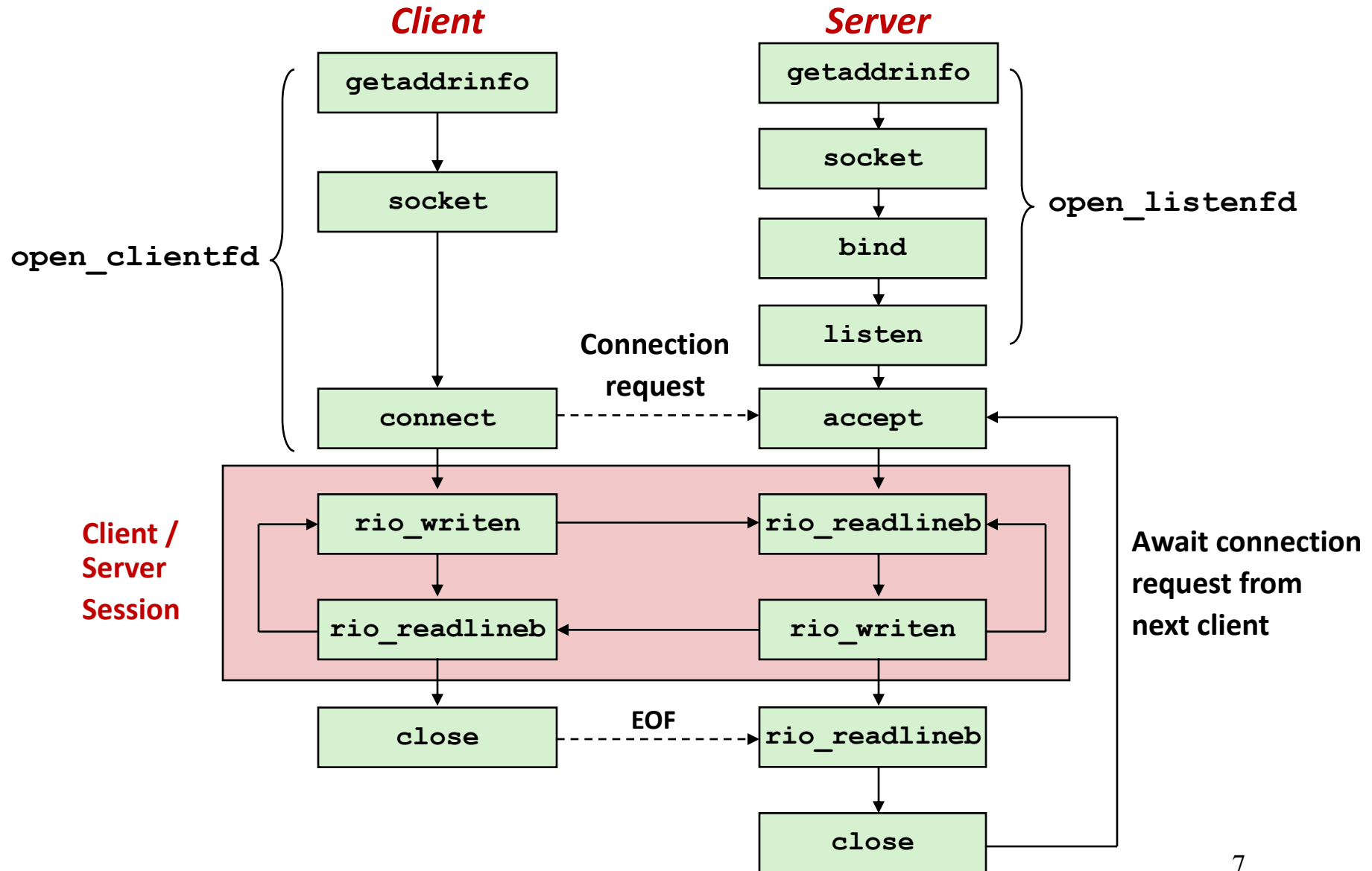
Concurrency

- Concurrent programming
 - Applications that use application-level concurrency
- Three basic approaches for concurrent programming
 - Processes
 - Threads
 - I/O multiplexing (Read the textbook by yourself)

Concurrency

- Concurrent programming is hard
 - The human mind trends to be sequential
 - Thinking about all possible sequences of events in a computer system is at least error prone and frequently impossible
 - Classical problem classes of concurrent programs
 - Data races, deadlock, and livelock/starvation

Iterative Echo Server



Review: Iterative Echo Server

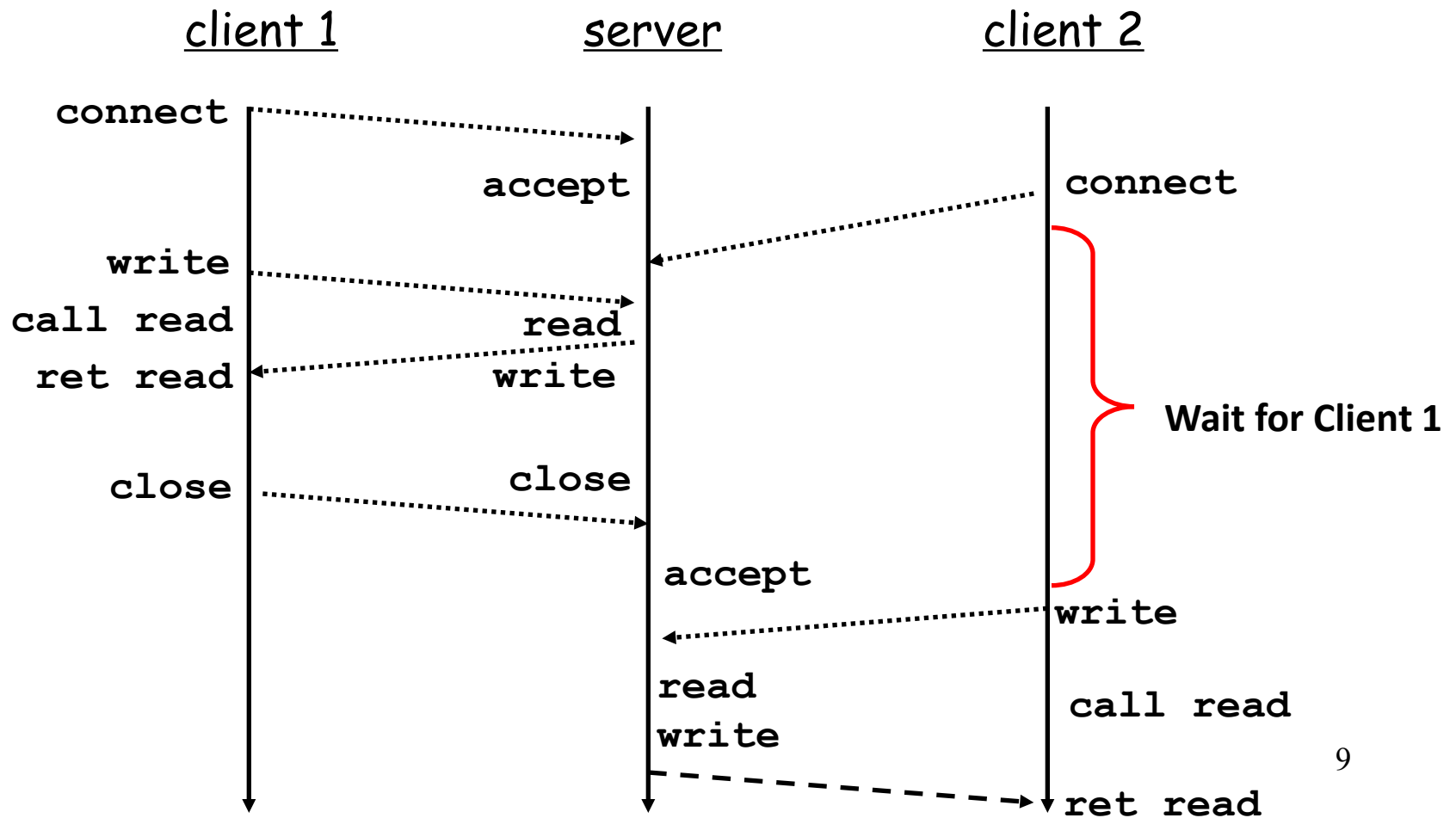
```
int main(int argc, char **argv)
{
    int listenfd, connfd;
    struct sockaddr_in clientaddr;
    socklen_t clientlen = sizeof(clientaddr);

    listenfd = Open_listenfd(argv[1]);
    while (1) {
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        echo(connfd);
        Close(connfd);
    }
    exit(0);
}
```

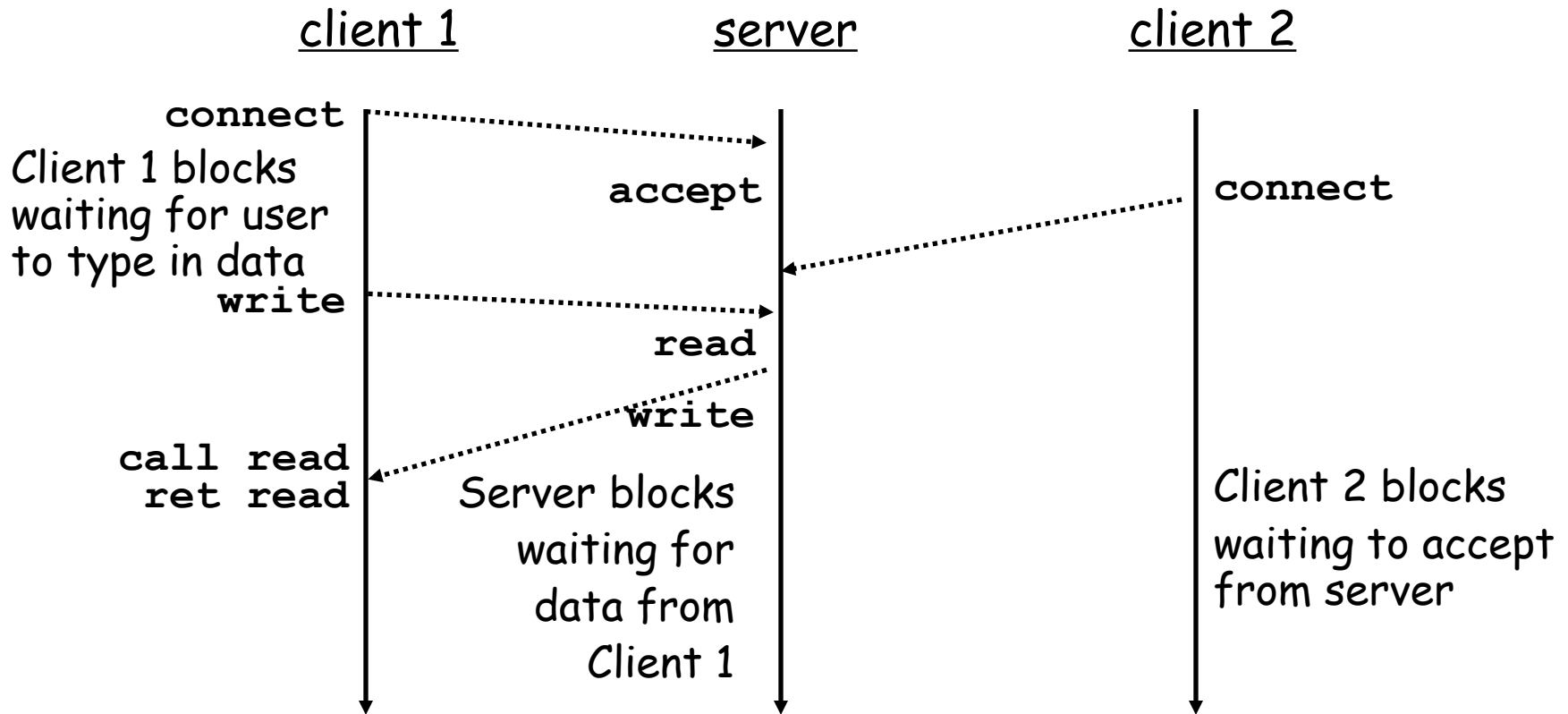
- Accept a connection request
- Handle echo requests until client terminates

Iterative Servers

- Iterative servers process one request at a time



Fundamental Flaw of Iterative Servers

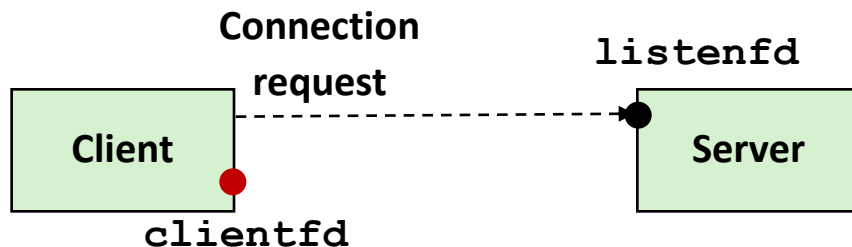


- Solution: use **concurrent** servers instead
 - Concurrent servers use multiple concurrent flows to serve multiple clients at the same time

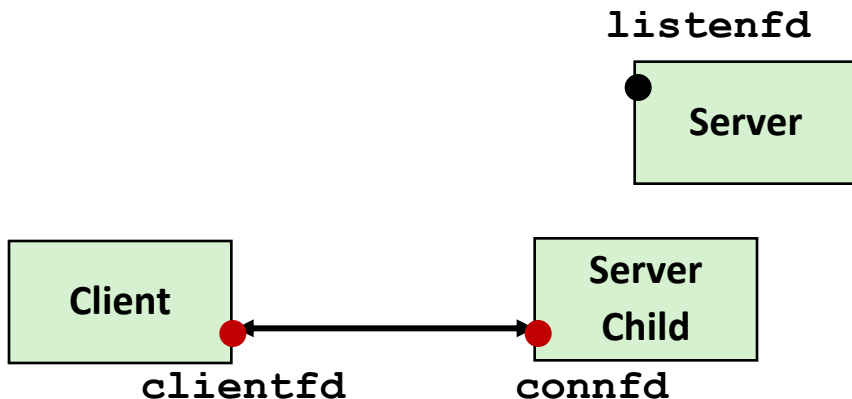
Concurrent Programming with processes



1. Server blocks in **accept**, waiting for connection request on listening descriptor `listenfd`

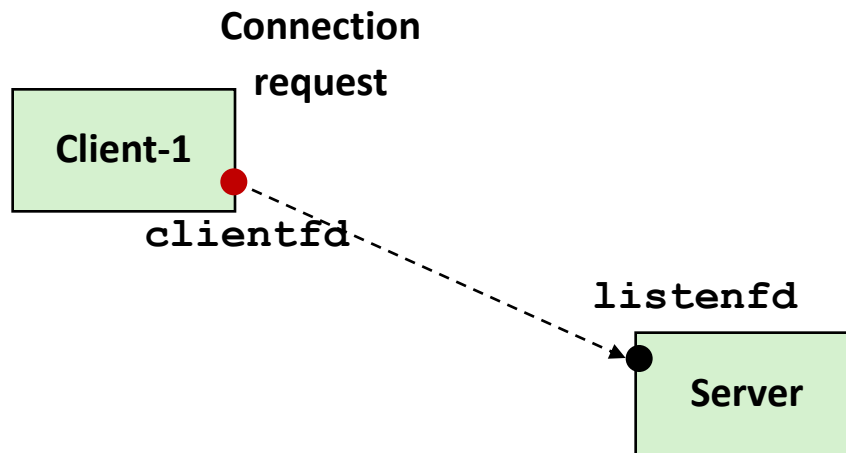


2. Client makes connection request by calling and blocking in **connect**

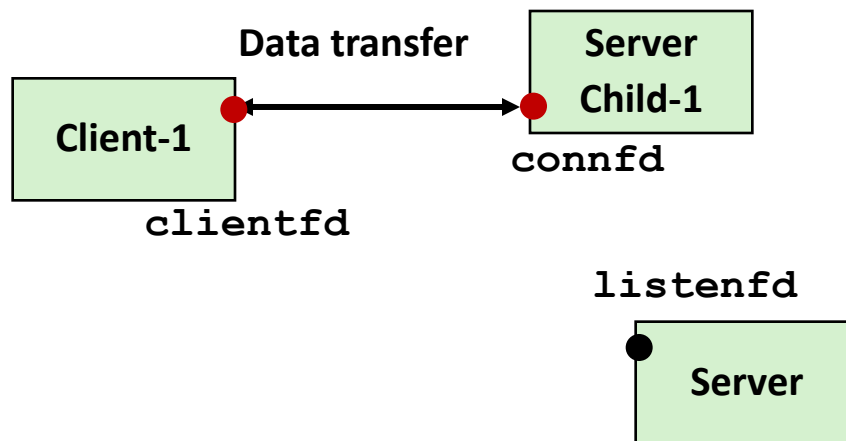


3. Server returns `connfd` from `accept` and **forks** child to handle client. Client returns from **connect**. Connection is now established between `clientfd` and `connfd`

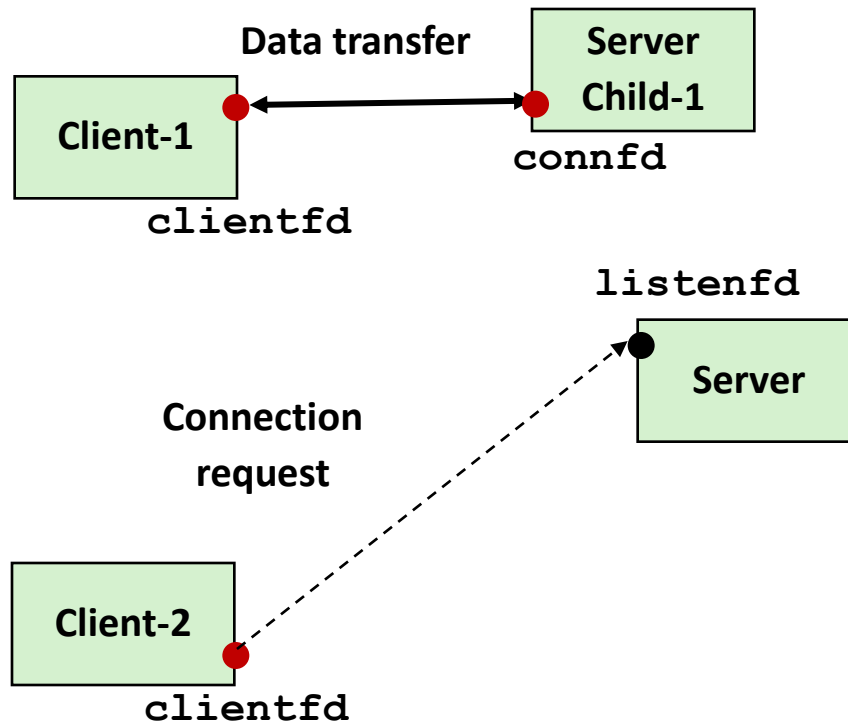
Concurrent Programming with processes



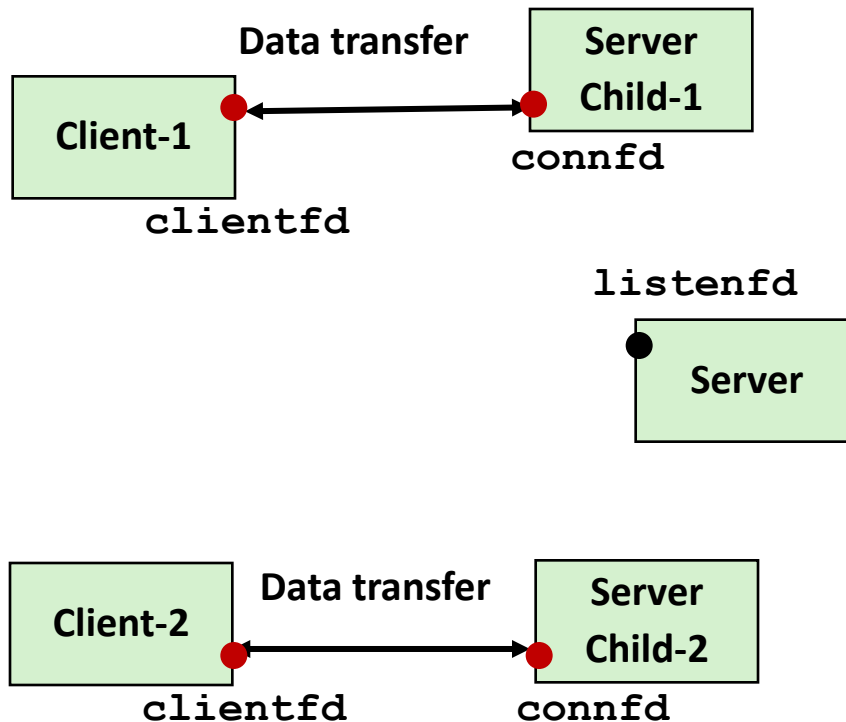
Concurrent Programming with processes



Concurrent Programming with processes



Concurrent Programming with processes



Process-Based Concurrent Server

```
int main(int argc, char **argv)
{
    int listenfd, connfd;
    struct sockaddr_in clientaddr;
    socklen_t clientlen = sizeof(clientaddr);

    Signal(SIGCHLD, sigchld_handler);
    listenfd = Open_listenfd(argv[1]);
    while (1) {
        connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
        if (Fork() == 0) {
            Close(listenfd); /* Child closes its listening socket */
            echo(connfd);    /* Child services client */
            Close(connfd);   /* Child closes connection with client */
            exit(0);         /* Child exits */
        }
        Close(connfd); /* Parent closes connected socket (important!) */
    }
}
```

□ Fork separate process for each client

Concurrent Programming with Processes

- Kernel provides multiple control flows with separate address spaces
- Standard Unix process control and signals
- Explicit interprocess communication mechanism

Implementation issues with process

- Server must reap zombie children
 - To avoid potential memory leak
 - Otherwise, OOM may happen

```
void sigchld_handler(int sig)
{
    while (waitpid(-1, 0, WNOHANG) > 0)
        ;
    return;
}
```

Implementation issues with process

- Server must close its copy of `connfd`
 - Kernel keeps reference for each socket
 - After fork, `refcnt(connfd) = 2`
 - Connection will not be closed until `refcnt(connfd)=0`

Pros and cons of process

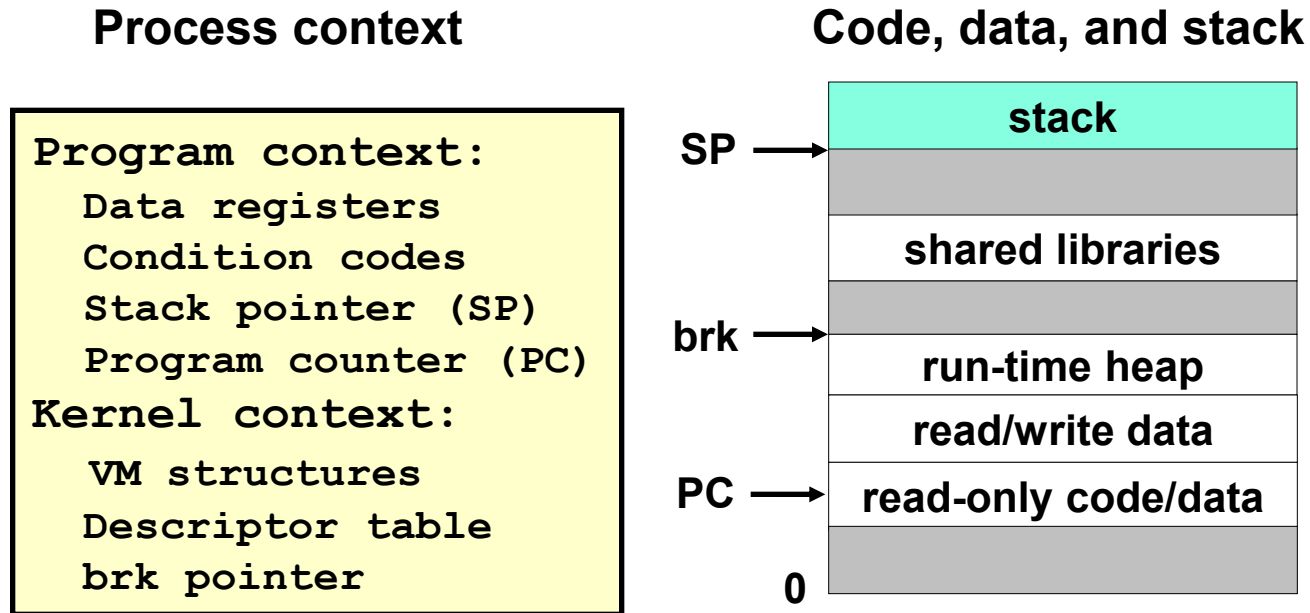
- + Handles multiple connections concurrently
- + Clean sharing model
 - descriptors (no)
 - file tables (yes)
 - global variables (no)
- + Simple and straightforward

Pros and cons of process

- - Additional overhead for process control
- - Nontrivial to share data between processes
 - Requires IPC (inter-process communication) mechanisms
 - E.g., how to accumulate the total bytes received from clients
 - FIFO's (named pipes), System V shared memory, etc.

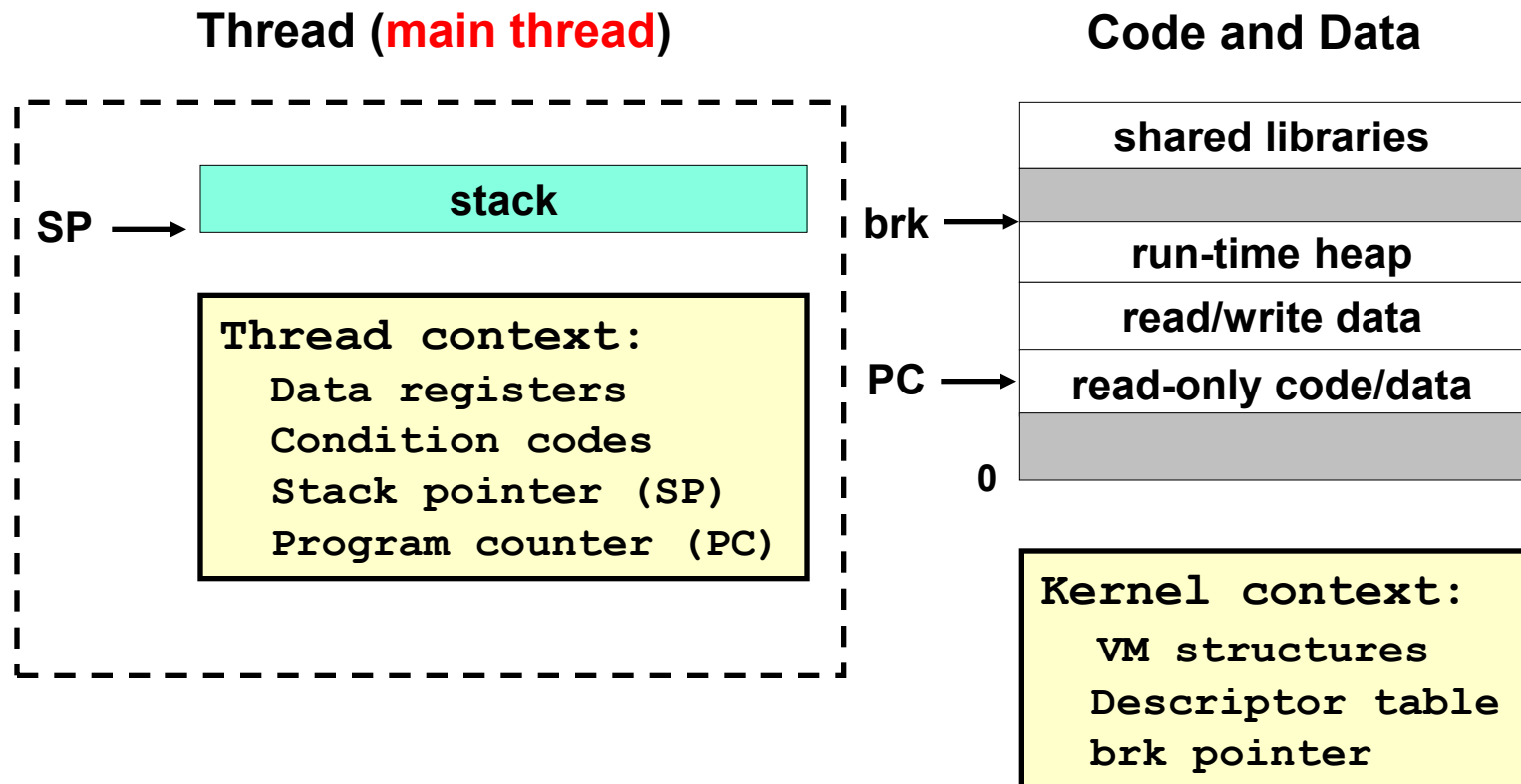
Traditional view of a process

- Process = process context + code, data, and stack



Alternate view of a process

- Process = thread + code, data, and kernel context



A process with multiple threads

- Multiple threads can be associated with a process
 - Each thread has its own logical control flow (sequence of PC values)
 - Each thread shares the same **code**, **data**, and **kernel context**
 - Each thread has its own **thread id** (TID)

A process with multiple threads

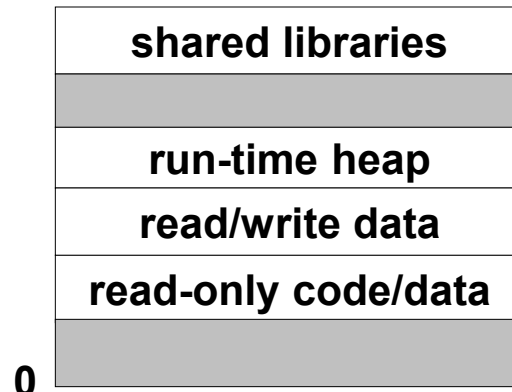
Thread 1 (main thread)

stack 1

Thread 1 context:

Data registers
Condition codes
SP1
PC1

Shared code and data



Kernel context:

VM structures
Descriptor table
brk pointer

Thread 2 (**peer thread**)

stack 2

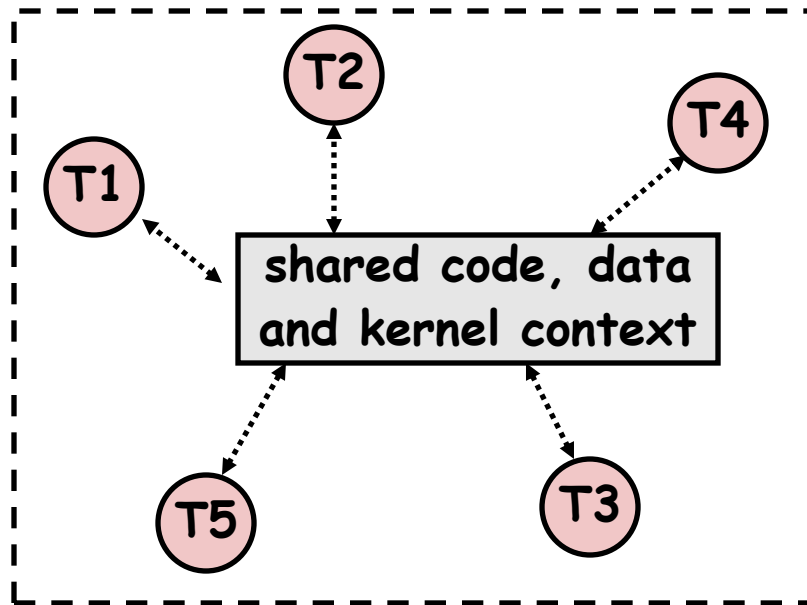
Thread 2 context:

Data registers
Condition codes
SP2
PC2

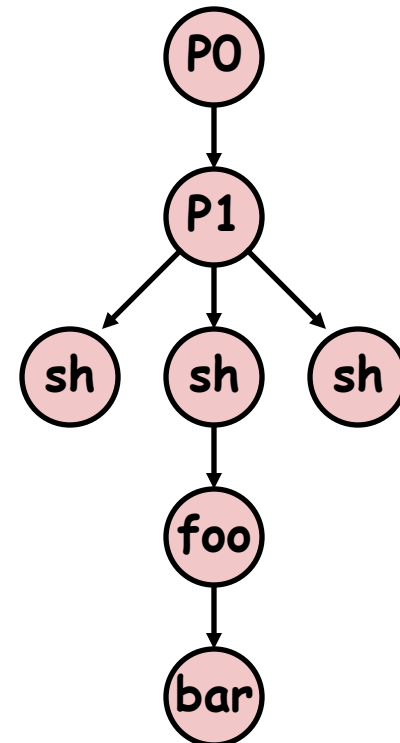
Logical View of Threads

- Threads associated with process form a pool of peers
 - Unlike processes which form a tree hierarchy

Threads associated with process foo



Process hierarchy



Posix threads (Pthreads) interface

- Pthreads: Standard interface for ~60 functions
 - Manipulate threads from C programs
 - Creating and reaping threads
 - `pthread_create`
 - `pthread_join`
 - Determining your thread ID
 - `pthread_self`
 - Terminating threads
 - `pthread_cancel`
 - `pthread_exit`
 - `exit` [terminates all threads]

The Pthreads "hello, world" Program

```
/* hello.c - Pthreads "hello, world" program */  
#include "csapp.h"
```

```
/* thread routine */
```

```
void *thread(void *vargp) {  
    printf("Hello, world!\n");  
    return NULL;  
}
```

Implicit pthread_exit

```
int main() {  
    pthread_t tid;
```

```
    Pthread_create(&tid, NULL, thread, NULL);
```

```
    Pthread_join(tid, NULL);
```

```
    exit(0);
```

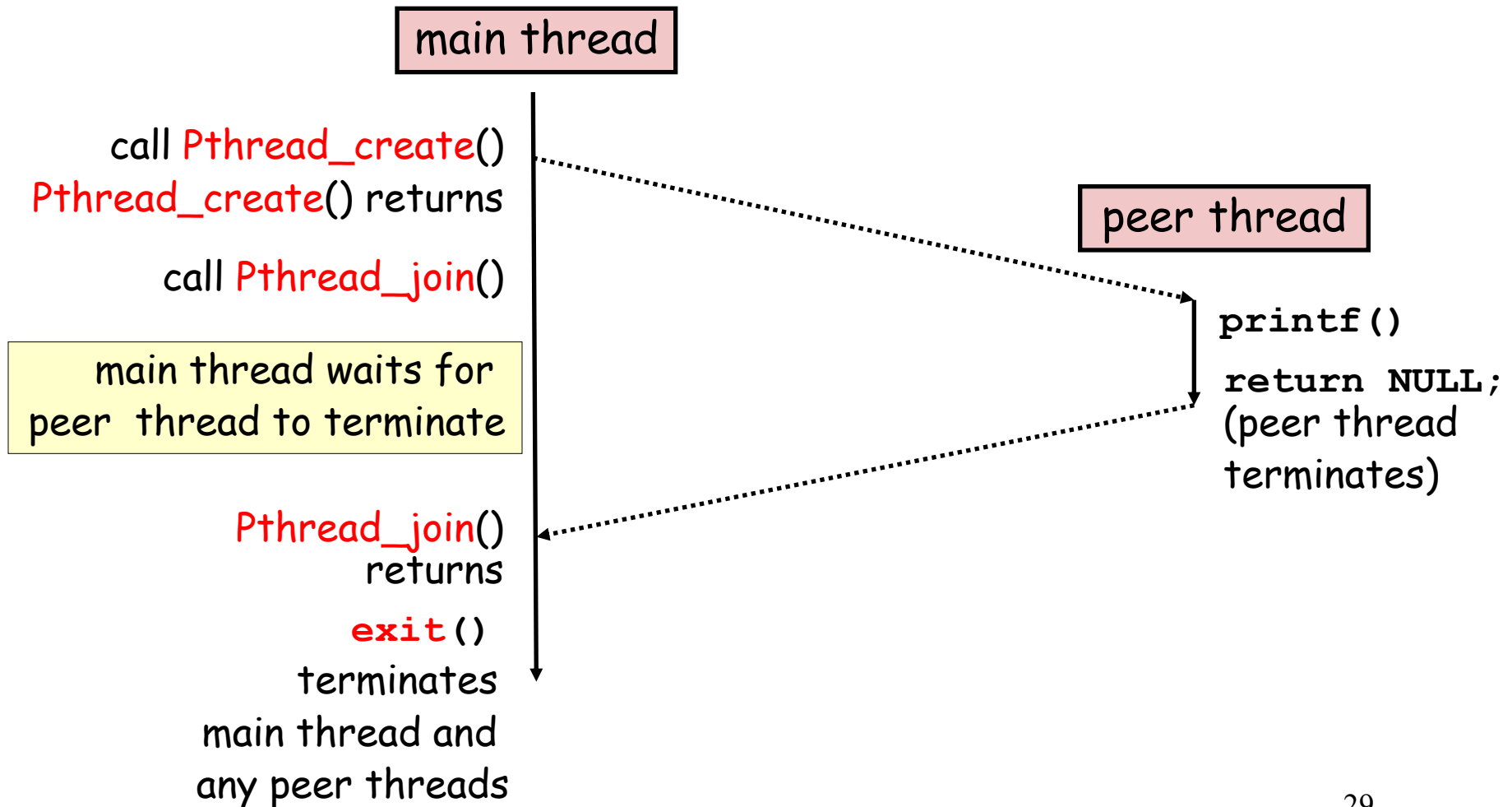
```
}
```

Thread attributes
(usually NULL)

Exit status of
the peer thread

Thread arguments
(void *vargp)

Execution of Threaded "hello, world"



Thread-based concurrent server

```
int main(int argc, char **argv){
    int listenfd, *connfdp;
    socklen_t clientlen;
    struct sockaddr_in clientaddr;
    pthread_t tid;

    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(0);
    }
    listenfd = open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(clientaddr);
        connfdp = Malloc(sizeof(int));
        *connfdp = Accept(listenfd,
                          (SA *)&clientaddr, &clientlen);
        Pthread_create(&tid, NULL, thread, connfdp);
    }
}
```

Thread-based concurrent server (cont)

```
/* thread routine */  
void *thread(void *vargp)  
{  
    int connfd = *((int *)vargp);  
  
    Pthread_detach(pthread_self());  
    Free(vargp);  
    echo(connfd);  
    Close(connfd);  
    return NULL;  
}
```

Issues with thread-based servers

- Must run “detached” to avoid memory leak
 - At any point in time, a thread is either **joinable** or **detached**
 - A joinable thread can be reaped and killed by other threads
 - must be reaped (with **pthread_join**) to free memory resources

Issues with thread-based servers

- Must run “detached” to avoid memory leak
 - Detached thread cannot be reaped or killed by other threads
 - resources are automatically reaped on termination
 - Default state is joinable
 - use `pthread_detach(pthread_self())` to make detached

Issues with thread-based servers

- Must be careful to avoid unintended sharing
 - For example, what happens if we pass the address of `connfd` to the thread routine?
 - `Pthread_create(&tid, NULL, thread, (void *) &connfd);`

Thread-based concurrent server (buggy)

```
int main(int argc, char **argv){
    int listenfd, connfd;
    socklen_t clientlen;
    struct sockaddr_in clientaddr;
    pthread_t tid;

    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(0);
    }
    listenfd = open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(clientaddr);
        connfd = Accept(listenfd,
            (SA *)&clientaddr, &clientlen);
        Pthread_create(&tid, NULL, thread, (void*)&connfd);
    }
}
```

Thread-based concurrent server (cont)

```
/* thread routine */  
void *thread(void *vargp)  
{  
    int connfd = *((int *)vargp);  
  
    Pthread_detach(pthread_self());  
    echo(connfd);  
    Close(connfd);  
    return NULL;  
}
```

Thread-based concurrent server (How about this)

```
int main(int argc, char **argv){
    int listenfd, connfd;
    socklen_t clientlen;
    struct sockaddr_in clientaddr;
    pthread_t tid;

    if (argc != 2) {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(0);
    }
    listenfd = open_listenfd(argv[1]);
    while (1) {
        clientlen = sizeof(clientaddr);
        connfd = Accept(listenfd,
            (SA *)&clientaddr, &clientlen);
        Pthread_create(&tid, NULL, thread, (void*)connfd);
    }
}
```

Thread-based concurrent server (cont)

```
/* thread routine */  
void *thread(void *vargp)  
{  
    int connfd = (int)vargp ;  
  
    Pthread_detach(pthread_self()) ;  
    echo(connfd) ;  
    Close(connfd) ;  
    return NULL ;  
}
```

Threads vs. Processes

- How threads and processes are **similar**
 - Each has its own logical control flow
 - Each can run **concurrently** with others (possibly on different cores)
 - Each is context switched

Threads vs. Processes

- How threads and processes are **different**
 - Threads share code and some data
 - Processes (typically) do not
 - Threads are somewhat **less expensive** than processes
 - Process control (creating and reaping) is twice as expensive as thread control
 - Linux numbers:
 - ~20K cycles to create and reap a process
 - ~10K cycles (or less) to create and reap a thread

Q: How about the perf data on your computer?

Pros and cons of thread-based designs

- + Easy to **share** data structures between threads
 - E.g., logging information, file cache
- + Threads are more **efficient** than processes
- - **Unintentional sharing** can introduce subtle and hard-to-reproduce errors!
 - The ease with which data can be shared is both the greatest strength and the greatest weakness of threads
 - Hard to debug with GDB